从进程镂空到模块踩踏：前言

前几章我们在用户态打转：PEB 遍历、哈希解析、IAT 隐藏、无 IAT 注入器。但从这一课开始，我们进入内存对抗的新阶段。第一站：进程镂空（Process Hollowing）的衰落，以及模块踩踏（Module Stomping）的崛起。本节不写代码，只讲清楚"为什么旧技术死了，新技术怎么来的"。

- **进程镂空是什么**

进程镂空是 2010 年代最流行的注入技术之一。它的核心思路是：

- 用 CreateProcess 启动一个合法进程（比如 notepad.exe），但以挂起方式启动
- 用 NtUnmapViewOfSection 把 notepad.exe 的内存映像整个卸载
- 用 VirtualAllocEx 在同一地址申请内存，把恶意 PE 写进去
- 修改线程上下文（SetThreadContext），让入口点指向恶意代码
- ResumeThread 恢复执行

结果：进程名是 notepad.exe，实际运行的是恶意代码。杀软看到进程名是 notepad.exe，不容易起疑。

为什么当年流行？

- 进程名合法，行为隐蔽。
- 内存里只有恶意代码，没有原始 notepad.exe 的痕迹。
- 配合 NtUnmapViewOfSection，干净利落。

- **为什么在现代 Windows 10/11 上失效了**

(一) 原因 1：ASLR（地址空间布局随机化）

ASLR 的物理动作：每次程序启动，Windows 会把 PE 文件的基址随机化。notepad.exe 这次加载在 0x7FF6A2B40000，下次可能在 0x7FF8C3D40000。

对进程镂空的影响：

- 进程镂空的核心步骤是"卸载原映像 → 在原地址申请内存 → 写入恶意 PE"。
- 但 ASLR 让"原地址"每次都不同。
- 必须先查询原进程的基址（通过 PEB），然后在那个地址申请内存。
- 如果申请失败（地址被占用），整个流程崩溃。

(二) 原因 2：ACG（Arbitrary Code Guard，动态代码防护）

ACG 的物理动作：Windows 10 引入的一项安全机制。它禁止进程把内存页从"可写"改成"可执行"。

对进程镂空的影响：

- 进程镂空最后需把 PE 的内存页从 PAGE_READWRITE 改成 PAGE_EXECUTE_READ。
- 如果目标进程开启了 ACG（很多现代 Windows 进程默认开启），VirtualProtectEx 会直接失败。
- 没有可执行权限，恶意代码就是一堆死字节。

(三) 原因 3：VirtualAllocEx 频繁返回 ERROR_INVALID_ADDRESS

物理原因：

- 当调用 NtUnmapViewOfSection 卸载原映像后，Windows 并不会立刻把那个地址空间完全释放。
- 那个地址周围可能还有其他映射（比如 PEB、TEB、堆、栈）。
- 调用 VirtualAllocEx 试图在原地址申请内存时，系统会检查"这个地址是否已被占用"。
- 如果周围有任何残留映射，申请就失败，返回 ERROR_INVALID_ADDRESS（错误码 487）。

结果：现代 Windows 10/11 上，进程镂空的成功率极低。

- **进程镂空的“替代方案”：模块踩踏**

既然“卸载整个进程映像再重建”太难了，那我们就不卸载，而是钻空子。

核心思路

不申请新内存，而是把 Shellcode 写进个已经加载的合法 DLL 的内存空间里，特别是它的 .text 节（代码段）中的空隙。

物理动作对比：

技术	内存操作	隐蔽性
进程镂空	卸载原映像 → 在原址申请新内存 → 写入 Shellcode	内存布局异常（原映像消失）
模块踩踏	找个已加载的合法 DLL → 在它的 .text 节空隙里写入 Shellcode	内存布局正常（DLL 还在）

- **模块踩踏的物理细节**

(一) 为什么 .text 节里有“空隙”

PE 文件的 .text 节在加载到内存时，会按照 SectionAlignment（通常是 4KB）对齐。但实际的代码长度往往不是 4KB 的整数倍，所以节区末尾会有填充空间（padding）。

举例：

- 某个 DLL 的 .text 节实际代码大小是 0x1A00 字节。
- 但内存对齐后占用 0x2000 字节。
- 中间的 0x600 字节就是空隙。

物理意义：这片空隙属于 .text 节，天然具有可执行权限（PAGE_EXECUTE_READ），且不会被 EDR 标记为“可疑新内存”。

(二) 模块踩踏的操作流程

- 选一个已加载的合法 DLL（比如 amsi.dll、winhttp.dll）
- 解析它的 PE 头，找到 .text 节的起始地址和大小
- 计算 .text 节内的空隙位置（节区末尾的 padding）
- 用 WriteProcessMemory 把 Shellcode 写入空隙
- 用 CreateRemoteThread 在空隙地址创建线程
- Shellcode 执行

关键优势：

- 不申请新内存：EDR 监控 VirtualAllocEx 时看不到任何动作。
- 不修改内存属性：.text 节天然可执行，不需要 VirtualProtectEx。
- 内存布局正常：DLL 的代码段看起来完好无损，只是末尾多了一小段“未知代码”。

- **模块踩踏的“可检测性”**

模块踩踏不是万能的，现代 EDR 也有对应的检测手段：

检测手段	原理	反制
内存完整性扫描	EDR 扫描已加载 DLL 的 .text 节，对比磁盘上的原始文件，发现被修改的字节	选择“不被频繁校验”的 DLL，或只修改 padding 区域
线程起始地址异常	EDR 监控 CreateRemoteThread，检查线程起始地址是否指向合法模块的入口	用 NtCreateThreadEx 替代，或者用回调函数（APC）执行
执行流异常	EDR 通过 ETW 监控代码执行，发现某段代码从未在磁盘上出现过	使用“代码洞”（Code Cave）而非 padding，或者只写入极短的跳转指令

- **模块踩踏 vs 进程镂空：对比总结**

对比项	进程镂空	模块踩踏
是否卸载原映像	是	否
是否申请新内存	是（VirtualAllocEx）	否（复用已有内存）
是否需要改内存属性	是（VirtualProtectEx）	否（.text 天然可执行）
ASLR 影响	严重	无
ACG 影响	严重	无
EDR 监控点	NtUnmapViewOfSection、VirtualAllocEx、VirtualProtectEx	WriteProcessMemory、CreateRemoteThread

- **本课核心结论**

➢ 进程镂空已死：ASLR、ACG、VirtualAllocEx 地址冲突三重打击下，它在现代 Windows 上成功率极低。

➢ 模块踩踏是替代方案：不卸载、不申请、不改属性，直接把 Shellcode 写进已加载 DLL 的 .text 节空隙。

➢ 模块踩踏的本质：利用合法模块的“可执行内存”，把恶意代码藏进合法代码里。

➢ 红蓝对抗的下回合：EDR 开始监控 .text 节完整性，红队需更隐蔽的“代码洞”技术。